HW 2B

A secure login for a Juice Shop-style shop

Secure feature design, a login form with checks on both sides, and an attempt to break that form.

Part 1. Secure feature design

1. SQL injection on login

Problem

A crafted email can change the database command and skip the password check, so someone signs in without the real password.

Fix

Bind the email with WHERE email = ?, so it stays data. Reject values that are not email addresses.

2. Cross-site scripting in search and reviews

Problem

A script planted in search or a review runs in the next shopper's browser and can use their signed-in session.

Fix

Write that input as text, and block inline scripts with a Content-Security-Policy.

3. Weak registration and password storage

Problem

A check that lives only on the page, or a weak reset secret, lets someone claim an account. A stolen database exposes plaintext or fast hashes.

Fix

The server decides identity. Store passwords with bcrypt.hashSync(password, 10). bcrypt.compareSync checks a guess without reversing the hash.

Part 2. Login form

This is an HTML, CSS, and JavaScript login, plus account creation. validateLogin blocks an empty submit, requires @ in the email, and requires a password of at least 8 characters. The browser runs that check, and the server runs it again. Passwords are stored with bcrypt at cost 10. After sign-in, the customer can order drinks. Run npm install, then npm start, and open http://127.0.0.1:47291. The sample account is demo@juice.lab / Lemonade1. README.md has the same steps. Publish this folder on GitHub and paste that public URL here. This copy has no public link yet.

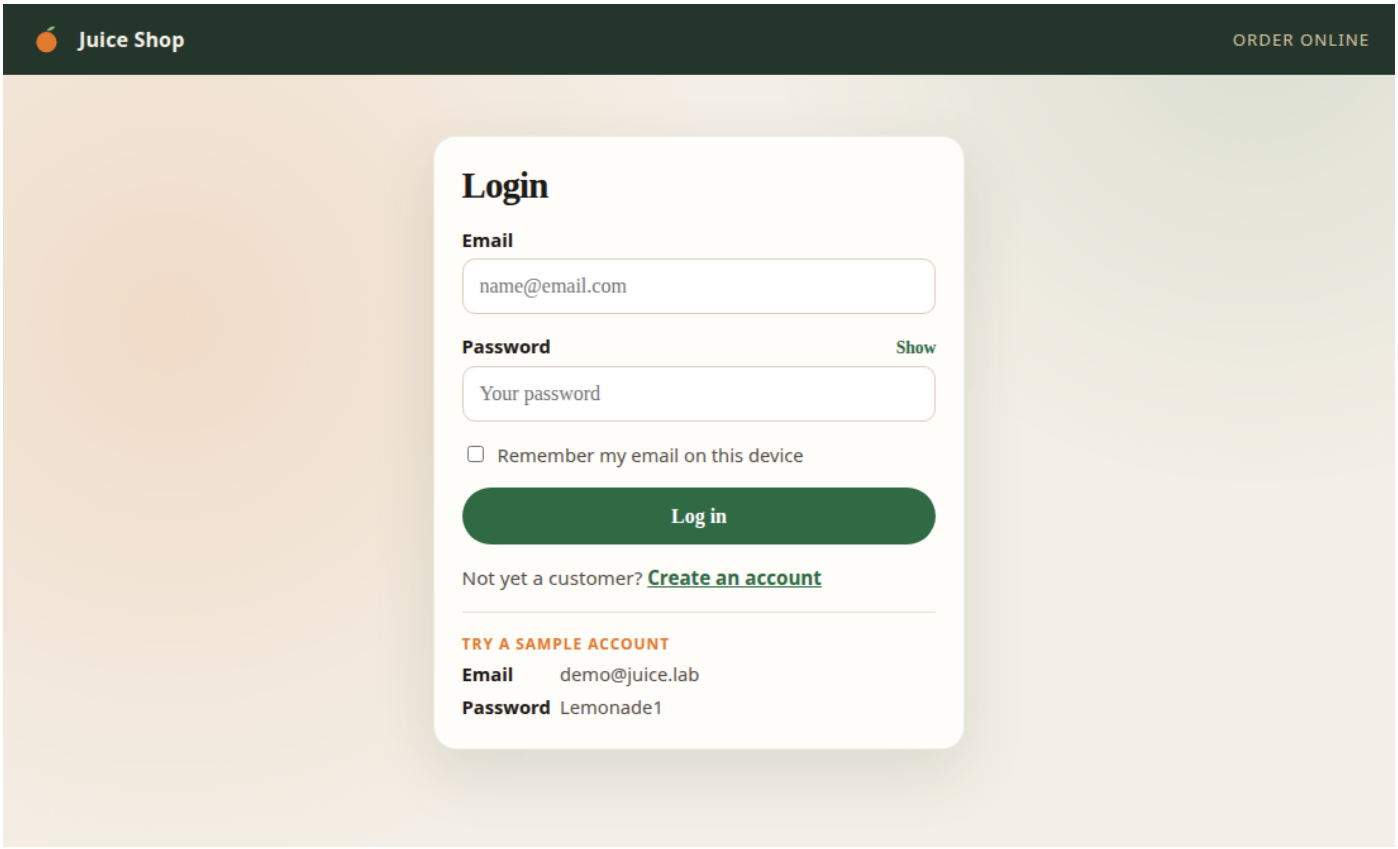


Figure 1. The sign-in card, with the sample account.

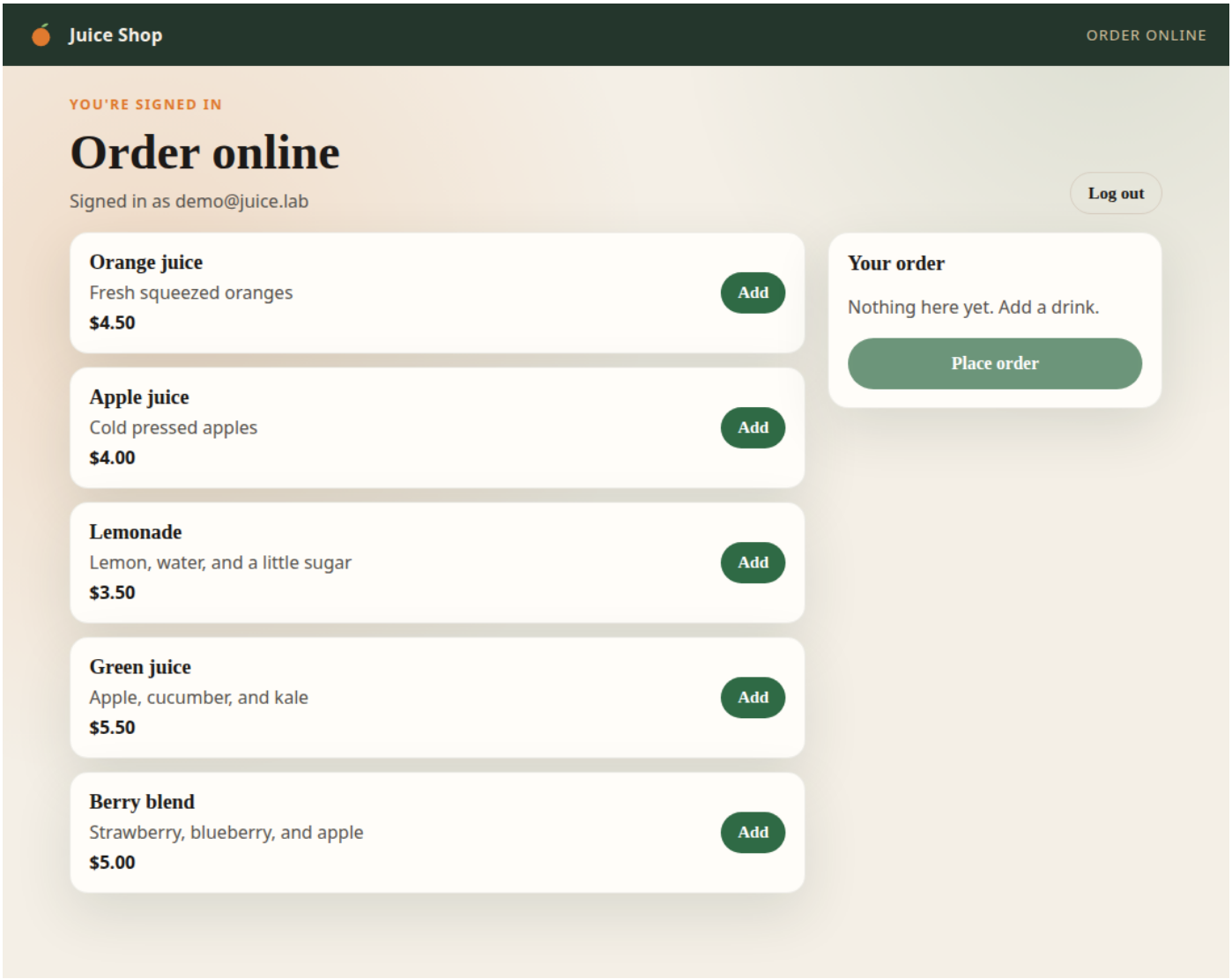


Figure 2. After sign-in, the customer can order drinks. Log out stays available.

Part 3. Trying to break the form

I tried to break this login at http://127.0.0.1:47291. An empty submit said the email and password are required. A short password was rejected for length. An email shaped like a database command was rejected with "That email or password doesn't match our records." The sample account stayed locked because the email is bound to ?. An email containing markup did not run a script. Those characters stayed in the box as text. The attack failed. The fix is the bound query and treating input as text. If that input were inserted as HTML, the browser could run it.

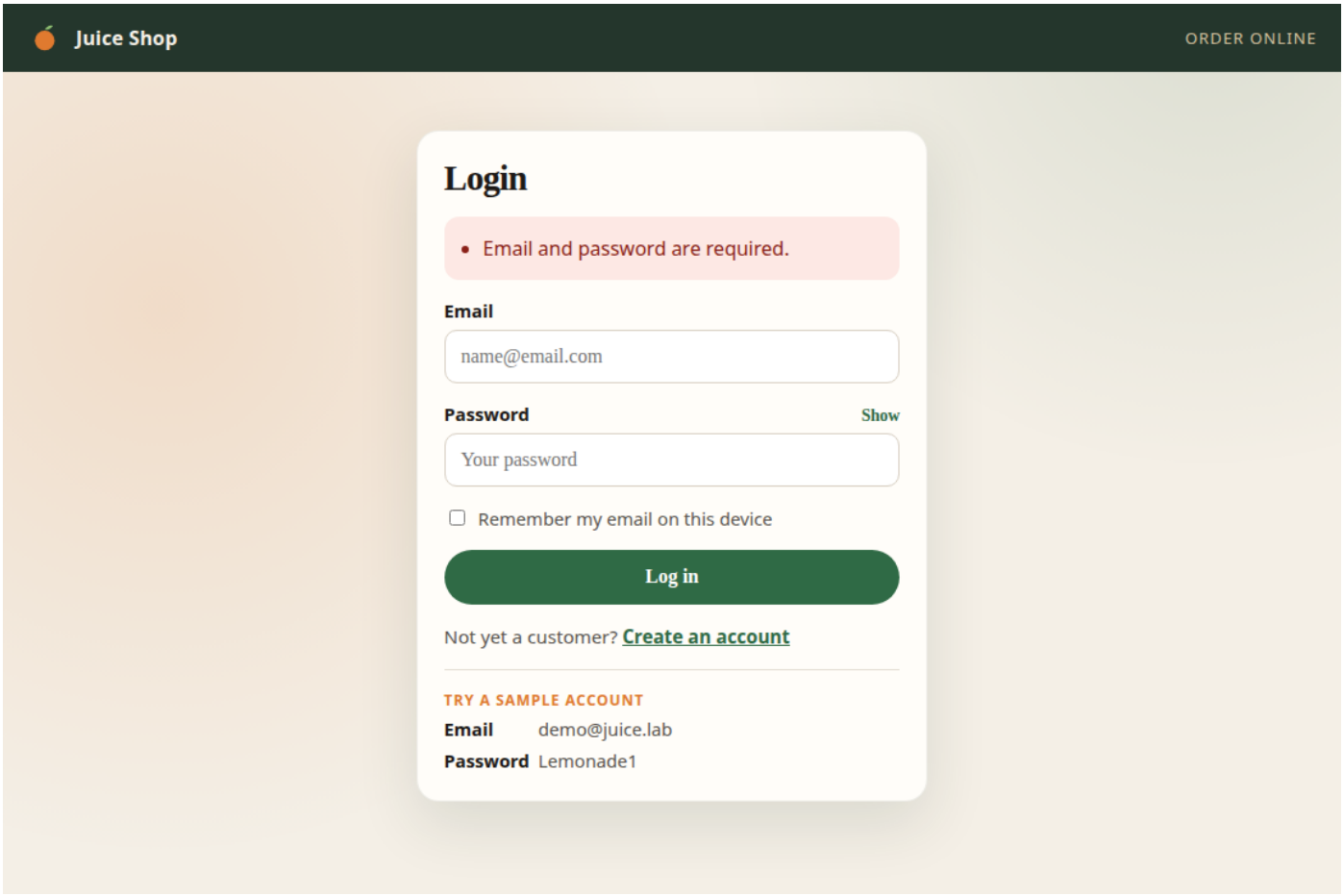


Figure 3. An empty submit is rejected.

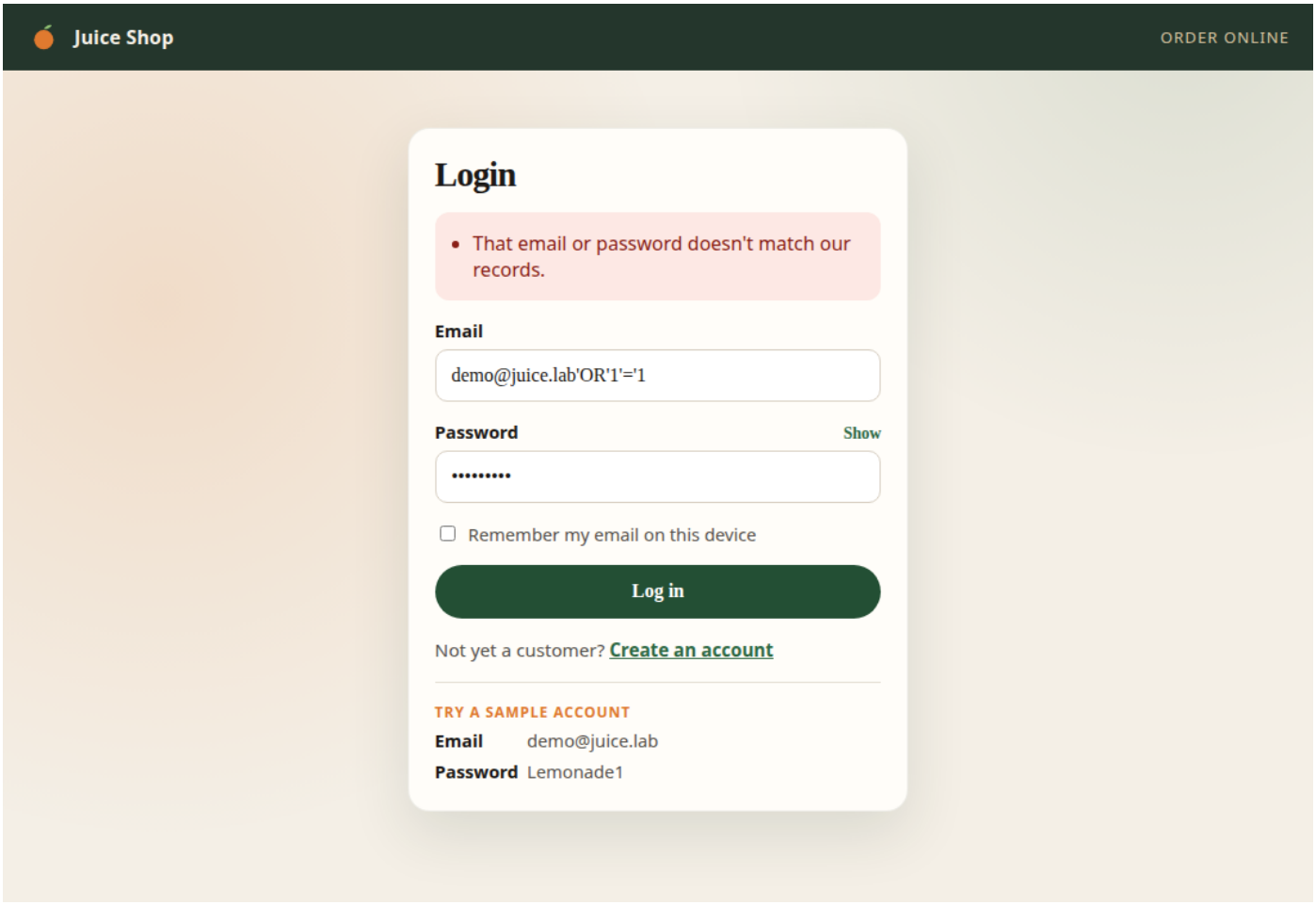


Figure 4. A database-shaped email is rejected. Sign-in does not succeed.

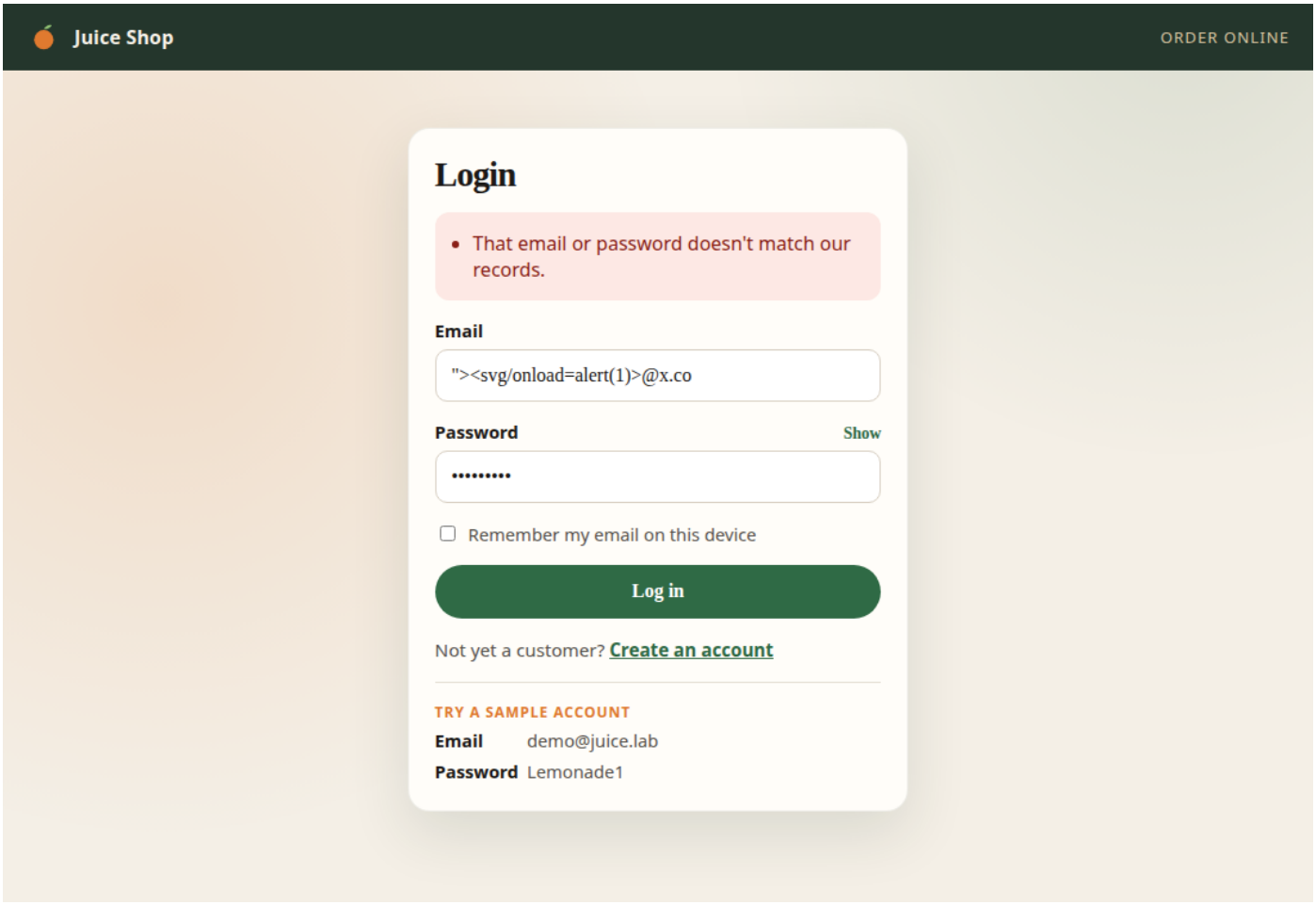


Figure 5. Markup in the email stays text. No script dialog ran.